

Ranked by Position: Order Sensitivity as an Exploitable Attack Surface in LLM Listwise Recommenders

Ge Zhang

Stanford University
gmzhang@stanford.edu

Jingru Cheng

Stanford University
cjr63@stanford.edu

Huiyuan Chen

Independent Researcher
hxc501@case.edu

Abstract

Large language models (LLMs) used as listwise rerankers in recommendation systems suffer from position bias when serializing candidate sets into prompts. We show this order sensitivity creates an exploitable attack surface: an attacker can promote a label-0 target into the top- k solely by reordering candidates, without changing item content, labels, or model parameters. We introduce $\text{promo}@k$ to quantify this vulnerability, measuring the fraction of label-0 targets that can be elevated into top- k rankings via permutation. Evaluating across three domains (MovieLens, Amazon Books, and Amazon Fashion), $\text{promo}@5$ reaches up to 0.57 at an attack budget of $R = 50$ orderings. Furthermore, ordinary permutation stability predicts vulnerability without running the attack. While a bidirectional T5 encoder scorer reduces exposure, permutation-consistency regularization and architectural invariance effectively mitigate it. Pointwise scoring avoids the bias issue but degrades ranking quality. These results demonstrate that input candidate order in listwise LLM reranking is a security-relevant attack vector. Code and data are available at https://github.com/geoz-lab/position_bias_attack.

1 Introduction

Large language models (LLMs) are increasingly used as listwise rerankers in recommendation and retrieval systems. Given a user history or query and a candidate set, a listwise reranker serializes all candidates into one prompt, scores them jointly, and returns a ranked list (Sun et al., 2023; Hou et al., 2024b; Pradeep et al., 2023b). This formulation enables cross-candidate comparison but creates a semantic–structural mismatch: candidate items form an unordered set, whereas decoder-only LLMs process them as an ordered token sequence. Through causal attention and positional

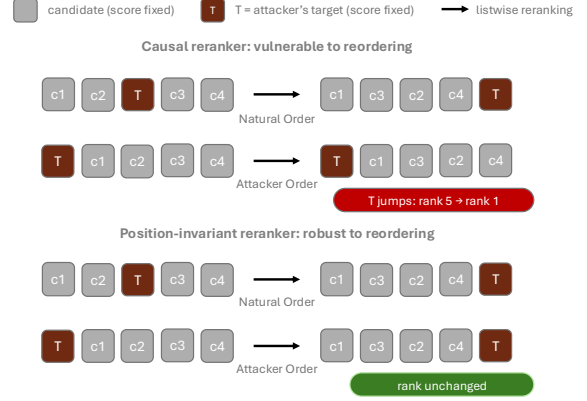


Figure 1: An ordering attack. The candidate set and its contents are fixed; only the *input order* changes. A causal LLM reranker (top) produces a different output for the attacker’s order, promoting a label-0 target T to the top; a position-invariant reranker (bottom) is unaffected.

encodings such as RoPE (Su et al., 2024), a candidate’s score may therefore depend on its position even when its content and relevance are unchanged.

Order sensitivity has been observed in in-context learning and LLM-based evaluation (Liu et al., 2024a; Wang et al., 2023; Shi et al., 2025). In reranking, prior work mainly treats it as instability to be measured or mitigated, for example through permutation averaging or position-robust training (Tang et al., 2024; Bito et al., 2026, 2025). We argue that this perspective understates the risk. Because candidate order is semantically arbitrary, systematic dependence on that order creates a manipulable input channel rather than merely random noise.

We therefore study candidate-order manipulation as an adversarial attack on listwise rerankers. The attacker changes only the presentation order of a fixed candidate set, without modifying item content, relevance labels, user behavior, or model

parameters (Figure 1). In our threat model, the client supplies the candidate ordering and may try a limited number of permutations, retaining the one that gives a chosen target its best rank (Section 3). We introduce $\text{promo}@k$ to measure the fraction of label-0 targets moved from outside the top- k into it through reordering alone. On the MovieLens anchor, trying 50 permutations manufactures top-5 exposure for 12% of targets under a causal reranker, compared with 0% under a position-invariant reference.

The vulnerability generalizes across eight causal rerankers and three recommendation domains. At an attack budget of $R = 50$, $\text{promo}@5$ reaches 0.57, with substantially larger exposure on the Amazon domains than on MovieLens. We find no evidence that exploitability decreases monotonically with model scale, while success increases smoothly with the attacker’s permutation budget. Moreover, ordinary permutation stability is strongly associated with attack severity across 24 model–domain settings, even after controlling for domain identity, providing an attack-free signal for auditing order-manipulation risk (Section 7).

We make three contributions. First, we formalize candidate-order manipulation as an attack on listwise LLM rerankers and introduce $\text{promo}@k$ to measure manufactured top- k exposure. Second, we show that the vulnerability is structured, budget-dependent, and present across eight rerankers and three recommendation domains, with no evidence of a monotonic reduction with model scale. Third, we identify permutation stability as an attack-free audit signal and evaluate four defense families, finding that consistency regularization substantially reduces exposure and architectural invariance removes the tested attack channel.

2 Related Work

LLM reranking and order sensitivity. LLMs have been widely studied as listwise rerankers for retrieval and recommendation, including RankGPT, open-source rerankers, and zero-shot recommendation methods (Sun et al., 2023; Pradeep et al., 2023a,b; Reddy et al., 2024; Hou et al., 2024b). These methods exploit joint cross-candidate comparison but generally treat the presented candidate order as fixed. Meanwhile, LLM outputs are known to vary with input position in long-context reasoning, evaluation, recommenda-

tion, and other structured tasks (Liu et al., 2024a; Xiao et al., 2023; Wang et al., 2023; Shi et al., 2025; Li et al., 2024; Jin et al., 2024; Bito et al., 2025; Chen et al., 2024). Prior work primarily frames such dependence as a capability, robustness, or evaluation issue. We instead study it as an attack surface. Because candidate sets are semantically unordered, an adversary may exploit their serialization order to manufacture top- k exposure.

Mitigating order sensitivity. Existing approaches reduce order dependence through architectural or encoding constraints (Bito et al., 2026; Yoon et al., 2024; Zhuang et al., 2024; Egressy and Stühmer, 2026), training-time regularization or calibration (Chao et al., 2024; Zhang et al., 2024; Zhao et al., 2025; Qiao et al., 2026; Joachims et al., 2017; Ai et al., 2018; Hager et al., 2024; Chen et al., 2025), and inference-time aggregation (Tang et al., 2024; Zeng et al., 2024). These methods are typically motivated by ranking quality, calibration, or stability. We evaluate representative defenses under an explicit order-manipulation threat model, treating permutation invariance as a security property. We also examine whether exploitability varies with model scale, rather than assuming that larger models are less position-sensitive.

Ranking manipulation attacks. Prior attacks manipulate rankings through prompts, candidate content, ranking criteria, or user interactions (Kumar and Lakkaraju, 2024; Tang et al., 2025; Liu et al., 2024b; Wang et al., 2024; Ning et al., 2024; Chen et al., 2022; Qian et al., 2025; Xing et al., 2026). Our attack is orthogonal to these approaches: it changes only the order of a fixed candidate set, while preserving item content, relevance labels, user behavior, and model parameters. This isolates candidate serialization as a distinct manipulation channel that prior content-based attacks often fix or control away.

3 Problem Setup and Threat Model

Listwise reranking. Given a user context H , such as an interaction history or search query, and a candidate set $\mathcal{C} = \{c_1, \dots, c_K\}$ with $K = 25$, a listwise reranker serializes the candidates according to an input order π , scores them in a shared prompt, and returns the candidates sorted by their scores. For our decoder-only rerankers, the score of a candidate is the mean token log-probability over the candidate’s text span. The temporal order of interactions inside H is preserved, as in standard

sequential recommendation; π refers only to the presentation order of the candidate items. Since $\mathcal{C}$ is a set once candidate content is fixed, permuting π should not change item relevance or the desired ranking, unless candidate order is explicitly intended as an input feature. We therefore treat dependence of the output ranking on π as unintended order sensitivity.

Causal and position-invariant rerankers. Our main reranker is a decoder-only causal model with standard causal attention and RoPE positional embeddings. We compare it with a position-invariant reranker based on InvariRank (Bito et al., 2026), which uses a structured cross-candidate attention mask and shared candidate position IDs to remove dependence on candidate presentation order. The two rerankers use the same scoring function, loss, and training data; they differ only in how cross-candidate attention and candidate positions are represented. We use the position-invariant reranker as a reference defense, not as a new architecture proposed by this work.

Permutation-consistency regularization. As a training-time defense, we add a consistency penalty that discourages the scorer from depending on candidate order. For each candidate set we sample P input orders $\pi_1, \dots, \pi_P$, score the set under each, and align the resulting score vectors back to canonical candidate identity. Writing $p^{(\pi)} = \text{softmax}(s^{(\pi)}/T)$ for the induced distribution over candidates, the penalty is the mean Kullback–Leibler divergence from the first order to each remaining one,

$$\mathcal{L}_{\text{perm}} = \frac{1}{P-1} \sum_{j=2}^P \text{KL}(p^{(\pi_1)} \| p^{(\pi_j)}), \quad (1)$$

and the training objective is $\mathcal{L} = \mathcal{L}_{\text{rank}} + \lambda_{\text{perm}} \mathcal{L}_{\text{perm}}$, where $\mathcal{L}_{\text{rank}}$ is the LambdaRank loss averaged over the same P orders. We use $P = 2$, $T = 1$, and $\lambda_{\text{perm}} = 1$. Gradients propagate through both branches: π_1 serves as the reference order but is not treated as a detached target, so the penalty pulls the two score distributions toward each other rather than distilling one into the other. The defense is approximate rather than exact, nothing in the architecture forbids order dependence at inference, which is why a small residual surface survives (Table 2).

Candidate labels. Candidate labels are derived from temporally held-out future interactions: observed ratings map to graded relevance labels 1–

4, while LightGCN-retrieved negatives and catalog filler items receive label 0. Label-0 candidates thus have no meaningful internal ground-truth order, and our attack metrics do not assume one is more relevant than another; full label rules and per-domain configurations are in Appendix A.1.

Threat model. The attacker aims to promote a chosen target item c^* into the top- k results by manipulating only the candidate presentation order. In our main evaluation, c^* is sampled from label-0 candidates, so the attack measures manufactured exposure for items that are label-0 under the held-out interaction labels. The attacker may control or influence the input order π , but cannot modify item content, relevance labels, user behavior, or model parameters.

We focus on a shared or API-based reranking service in which the client supplies an ordered candidate list and observes the returned ranking. The attacker is budget-limited: for each target, it may try at most R candidate orderings and keep the ordering that gives the target its best rank. This models a black-box ordering attack that uses only ranking outputs.

Exploitability metrics. For an evaluation instance i , let c_i^* denote the target item and let $r_{i,\pi}(c_i^*)$ be the output rank of the target under candidate ordering π , where rank 0 is best. We draw a pool of R_{max} random orderings $\Pi_{R_{\text{max}}} = (\pi_1, \dots, \pi_{R_{\text{max}}})$ and write $\Pi_R = (\pi_1, \dots, \pi_R)$ for its first R elements, so that $\Pi_1 \subseteq \dots \subseteq \Pi_{R_{\text{max}}}$. The attacker’s budget is R ; the pool size R_{max} is a property of the measurement rather than of the attacker.

We summarize the target’s typical, attack-free behavior by its mean rank over the full pool,

$$\bar{r}_i(c_i^*) = \frac{1}{R_{\text{max}}} \sum_{\pi \in \Pi_{R_{\text{max}}}} r_{i,\pi}(c_i^*), \quad (2)$$

and define its adversarial rank at budget R as the best rank achieved by any ordering the attacker tried,

$$r_i^{\text{adv}}(c_i^*) = \min_{\pi \in \Pi_R} r_{i,\pi}(c_i^*). \quad (3)$$

For brevity we write $\bar{r}_i$ and r_i^{adv} when the target is clear from context. Fixing $\bar{r}_i$ at the full pool keeps the promotion criterion below independent of R , which makes $\text{promo}@k$ monotonically non-decreasing in the attacker’s budget. Unless otherwise noted we use $R_{\text{max}} = R = 50$.

We report **rank gain**, the number of positions the target gains under attack $(\bar{r}_i - r_i^{\text{adv}})$, and

Into@5, the fraction of targets reaching the top five under at least one attempted ordering ($r_i^{\text{adv}} < 5$), as a supporting diagnostic. Unlike promo@5 below, Into@5 does not require the target to start outside the top five.

Third, promo@ k measures newly manufactured top- k exposure. Over the N evaluation instances we define

$$\text{promo}@k = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\bar{r}_i \geq k > r_i^{\text{adv}}], \quad (4)$$

the fraction of all targets that lie outside the top- k under attack-free ordering *and* can be driven inside it by reordering alone. Unlike Into@ k , it excludes targets already ranked inside the top- k without an attack. Because the denominator is the full evaluation set rather than an eligibility-filtered subset, promo@ k is a population-level exposure rate and is directly comparable across models and domains.

Permutation stability. To audit order sensitivity without running the adversarial search, we measure ordinary permutation stability. For each candidate set, we sample random candidate permutations, obtain one output ranking per permutation, and compute Kendall’s τ between pairs of output rankings. We average τ over permutation pairs and candidate sets. Higher τ means that the reranker produces more stable rankings under candidate reshuffling; lower τ indicates stronger dependence on presentation order.

Experimental setup. We fine tune rerankers with LoRA ($r = 16$, $\alpha = 32$) for 500 steps, using sequence length 4096 and LambdaRank loss. Experiments cover three domains: MovieLens-32M (Harper and Konstan, 2015) and two Amazon Reviews categories (Hou et al., 2024a), Books and Fashion. Each candidate set contains $K = 25$ items. We initialize candidate sets with LightGCN (He et al., 2020) first-stage retrieval, insert the selected held-out future items used to define relevance labels, and add catalog filler items when needed to keep the candidate-set size fixed. This insertion step is important for the sparse Amazon domains, where LightGCN recall@100 is only about 1%.

For MovieLens and Amazon Books, we use a history window of 20 interactions. For Amazon Fashion, we use a shorter history window of 5 interactions to retain enough eligible users under its sparser interaction histories. Our anchor

model is Llama-3.2-3B, evaluated with three seeds on MovieLens. We additionally sweep causal-only Qwen3 models from 0.6B to 14B, Llama-3.1-8B, and Mistral-7B across all three domains. We also include a bidirectional T5 encoder scorer as a mechanism comparison. Following the encoder-based design of ListT5 (Yoon et al., 2024), it encodes the serialized candidate list bidirectionally, but it is trained by us rather than taken from released weights: a flan-t5-base backbone with LoRA ($r = 32$) for 2,000 steps at learning rate 10^{-3} , scoring candidates from pooled hidden states under a softmax cross-entropy ranking loss. We therefore describe it as ListT5-style rather than as ListT5 itself.

Evaluation sample sizes depend on the metric. Standard ranking effectiveness and permutation-stability metrics, including nDCG@10 and Kendall’s τ , use 2,000 candidate sets. Position-sensitivity and headline attack analyses use 300 candidate sets; headline attack results are reported at budget $R = 50$, and the budget curve evaluates $R \in \{1, 5, 10, 20, 50\}$. For the attack analyses we sample one label-0 target per candidate set, giving $N = 300$ evaluation instances per model–domain cell. Additional baselines and defense ablations, including pointwise scoring and test-time order averaging, are reported in the corresponding tables and appendix.

All anchor attack metrics—the mechanism comparison, defense ablations, prompt ablations, and the attack-budget curve—are computed from a single reference run with $R_{\text{max}} = 50$ on the MovieLens anchor (seed 42), so that values are directly comparable across tables.

4 Position Bias Is Structured

We first ask whether candidate-position effects are systematic or merely random noise. For a fixed target candidate c^* , we keep its content and relevance label unchanged, move it through each input position $0, \dots, K - 1$, keep the remaining candidate set fixed, and record the target’s output rank. We aggregate results over candidate sets and stratify by relevance label.

Figure 2 shows a structured position-effect curve on the MovieLens anchor. The causal reranker exhibits a clear positional sweet spot: label-0 targets receive better output ranks in early-middle input positions, while the first and tail positions are worse. By contrast, the position-invariant

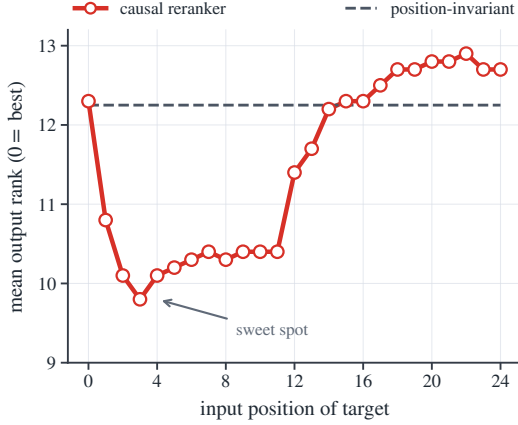


Figure 2: **Position bias is structured.** Mean output rank of a fixed target candidate as its input position is varied. The causal reranker exhibits a clear positional sweet spot, while the position-invariant reranker is nearly flat.

reranker is nearly flat. We summarize this effect using the position-curve range, defined as the difference between the worst and best mean output rank across input positions. For label-0 targets, the range is 3.09 for the causal reranker but only 0.06 for the position-invariant reranker. The resulting positional sweet spots make candidate order predictable enough to search adversarially.

5 Exploitability

Attacker. We use budget-limited random-order search. For each target c^* , the attacker samples R candidate orderings and keeps the ordering that gives c^* its best output rank. Unless otherwise noted, headline attack results use $R = 50$. We use black-box random search over at most R permutations and retain the ordering yielding the targets best rank.

Figure 3 reports the main exploitability result using promo@5 . On the MovieLens anchor, the causal reranker is vulnerable to order manipulation: changing only candidate order yields a rank gain of 3.34 positions, moves the target into the top five in 27.3% of cases, and manufactures new top-5 exposure for 12.0% of label-0 targets. By contrast, the position-invariant reranker has $\text{promo@5} = 0.000$. Thus, for the causal reranker, input order alone can create top-5 exposure for label-0 targets that would otherwise remain outside the top five.

The attack surface is substantially larger on the Amazon domains (Figure 3); the position-invariant reference stays at or near zero throughout.

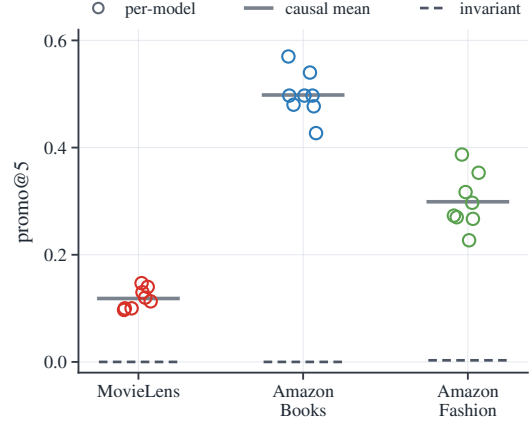


Figure 3: **Manufactured exposure across domains.** Each point is one causal model; horizontal bars denote domain means; dashed lines show the position-invariant reference.

Weaker content signals appear to make candidate position a stronger lever.

6 Mechanism Comparisons and Defenses

Encoder and pointwise controls. Table 1 compares the causal reranker with three controls on the MovieLens anchor. The T5 encoder scorer encodes the serialized candidate list bidirectionally rather than with decoder-only causal attention, but it still uses candidate order and positional information. Pointwise scoring removes candidate-order dependence by scoring each candidate independently, but gives up listwise comparison and requires $O(K)$ scoring.

The T5 encoder scorer improves ranking quality and reduces order exploitability, but does not eliminate it: promo@5 drops from 0.120 to 0.057, and the label-0 position-curve range drops from 3.09 to 0.71. Architectural invariance and pointwise scoring remove manufactured exposure, with $\text{promo@5} = 0$. The pointwise reference still has $\text{Into@5} = 0.157$, meaning that some label-0 targets can receive top-5 exposure because of fixed content-based model errors; however, its $\text{promo@5} = 0$ and zero rank gain show that this exposure is not manufactured by candidate-order manipulation. Because the T5 scorer also differs in backbone, loss, and scoring head, we interpret it as a mechanism comparison rather than evidence that bidirectionality alone causes the reduction.

Defense results. Table 2 summarizes the main defense results across domains. Naive order augmentation is not sufficient: on the MovieLens an-

Table 1: **Mechanism comparison on the MovieLens anchor.** Range denotes the label-0 position-curve range.

Model	Structure	nDCG@10	Range	τ	Gain	Into@5	promo@5
Causal Llama-3B	causal decoder	0.829	3.09	0.845	3.34	0.273	0.120
T5 encoder scorer	bidirectional encoder	0.876	0.71	0.913	1.88	0.197	0.057
Architectural invariance	constrained listwise	0.784	0.06	0.992	0.22	0.157	0.000
Pointwise reference	isolated scoring	0.763	0.00	1.000	0.00	0.157	0.000

chor, training on permuted candidate orders leaves promo@5 nearly unchanged at 0.117 as shown in Table 7, compared with the baseline value of 0.120. In contrast, permutation-consistency regularization substantially reduces the attack surface, lowering promo@5 from 0.120 to 0.007 on MovieLens, from 0.480 to 0.067 on Books, and from 0.297 to 0.047 on Fashion. Exact architectural invariance closes the residual surface, with promo@5 = 0.000 on MovieLens and Books and 0.003 on Fashion.

The quality–robustness trade-off is domain-dependent: on Books, exact invariance zeroes promo@5 at the largest nDCG@10 cost, while permutation-consistency regularization retains more quality for a still-large reduction (Table 2). Costs are smaller on MovieLens and Fashion.

7 Generalization

We evaluate whether order exploitability generalizes beyond the MovieLens anchor using eight causal rerankers across MovieLens-32M, Amazon Books, and Amazon Fashion. The sweep includes Qwen3 models from 0.6B to 14B parameters, Llama-3.1-8B, Mistral-7B, and Llama-3.2-3B. Unless noted otherwise, we report promo@5 at $R = 50$; full results appear in Appendix Table 9.

Model scale and domain severity. Figure 4 shows no monotonic decrease in exploitability with model size. Across the Qwen3 sweep, promo@5 ranges from 0.10–0.14 on MovieLens, 0.23–0.39 on Amazon Fashion, and 0.43–0.57 on Amazon Books. Seed variation on the MovieLens anchor is comparable to or larger than the differences among backbones, so small within-domain gaps should not be over-interpreted. The much larger cross-domain differences nevertheless indicate that domain characteristics influence exploitability more strongly than parameter count.

This domain pattern is consistent with weaker content signals in the Amazon datasets. MovieLens provides structured metadata, whereas the

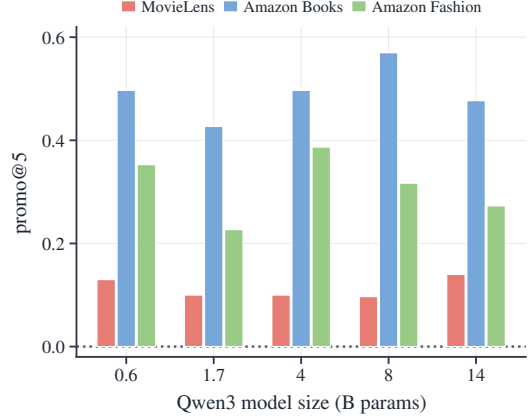


Figure 4: **Vulnerability does not monotonically decrease with model size.**

Amazon domains are sparser and have LightGCN Recall@100 of about 0.013. This association suggests that position may become a stronger cue when content evidence is weak, but it does not establish causality. The position-invariant reference remains at or near zero across domains, although its ranking-quality cost is largest on Books (Table 2).

Permutation stability as an audit signal. To test whether vulnerability can be audited without adversarial search, we correlate Kendall’s τ under random candidate reshuffling with promo@5 across the 24 causal model–domain cells.

Figure 5 shows a strong negative association between permutation stability and exploitability ($r = -0.974$, 95% bootstrap CI $[-0.989, -0.956]$). The relationship remains substantial after controlling for domain identity ($r = -0.776$, $p = 8 \times 10^{-6}$), indicating that stability is not merely a proxy for domain difficulty. Permutation stability therefore provides a practical, attack-free audit signal for order exploitability.

Attack budget sensitivity. We also vary the attack budget on the MovieLens anchor. For the causal reranker, promo@5 rises from 0.023 at $R = 1$ to 0.120 at $R = 50$; the T5 encoder follows the same trend at lower magnitude, reaching

Table 2: **Cross-domain defense landscape.** Lower promo@5 is better. Exact indicates whether the method is invariant by construction.

Defense / model	MovieLens		Amazon Books		Amazon Fashion		Exact
	nDCG@10	promo@5	nDCG@10	promo@5	nDCG@10	promo@5	
Baseline causal	0.829	0.120	0.618	0.480	0.449	0.297	No
Architectural invariance	0.784	0.000	0.510	0.000	0.424	0.003	Yes
Permutation-consistency KL	0.777	0.007	0.552	0.067	0.424	0.047	Approx.
Pointwise reference	0.763	0.000	–	0.000	–	0.000	Yes

Pointwise nDCG is measured on the MovieLens anchor only. Its cross-domain promo@5 = 0 follows by construction because candidates are scored independently.

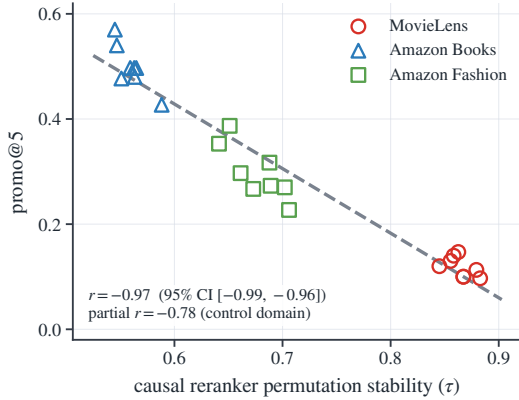


Figure 5: **Permutation stability tracks order exploitability.** Kendall’s τ versus promo@5 across 24 causal model–domain cells. Lower stability is associated with greater manufactured exposure.

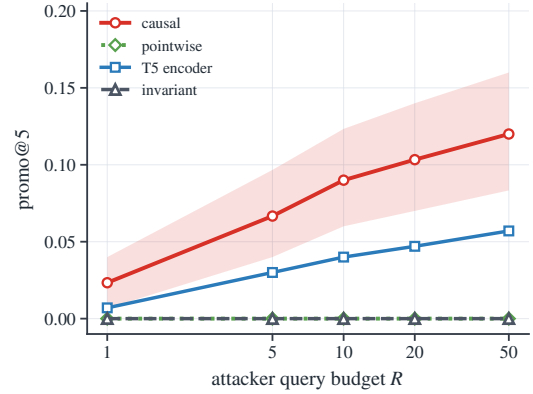


Figure 6: **Attack success increases with query budget.** promo@5 grows with the number of attempted orderings for order-sensitive scorers, while the position-invariant and pointwise references remain at zero. Shading indicates 95% bootstrap confidence intervals.

0.057. The position-invariant and pointwise references remain at zero, showing that the budget effect is specific to order-sensitive scoring.

8 Conclusion

Candidate order is a manipulable attack surface to listwise LLM rerankers. Across models and domains, reordering alone promotes label-0 targets, with greater exposure under larger query budgets and less stable rerankers. Permutation stability therefore provides a practical audit signal. Consistency regularization mitigates most exposure, while architectural invariance removes the tested attack channel, subject to a domain-dependent ranking-quality cost. Listwise rerankers should treat permutation robustness as a security requirement rather than only a stability property. The same audit principle may apply to other systems that serialize semantically unordered alternatives, although this remains to be tested.

Limitations

First, our threat model assumes the adversary can manipulate candidate presentation order, which aligns with API-based or shared reranking services but less so with systems using fixed or hidden candidate assembly. By evaluating single-stage reranking without altering retrieval pools, item content, or model parameters, we isolate candidate order as an attack channel, though we do not capture complex multi-stage attacks. Additionally, because label-0 targets are defined relative to held-out interaction data, promo@ k measures manufactured exposure for non-positive evaluation items rather than intrinsic user irrelevance. Furthermore, our budget-limited random search provides a conservative lower bound on exploitability, as adaptive or optimized search strategies could uncover more effective orderings.

Second, our experimental scope and analytical claims carry specific boundaries. While primary defense families are evaluated across domains, full architectural and training ablations are centered on

our anchor setting; additionally, pointwise scoring serves as an invariant baseline rather than a direct drop-in replacement, given its distinct computational profile. Finally, the observed relationship between Kendall’s τ and promo@5 across our 24 model–domain settings is observational rather than causal. Consequently, we position permutation stability as a practical, attack-free auditing signal rather than a proven causal mechanism of exploitability.

Ethics Statement

This work is intended to support auditing and mitigation of order-based vulnerabilities in LLM rerankers. We report aggregate vulnerability magnitudes, evaluate defenses, and avoid releasing an optimized order-manipulation pipeline. The attack studied here changes only candidate order and is therefore useful for isolating the positional channel; it should not be interpreted as guidance for manipulating deployed ranking systems. The content-edit diagnostic, if used, is limited to separating positional effects from content effects and is not designed as a competitive content attack. By identifying effective mitigations such as architectural invariance and permutation-consistency regularization, our goal is to help practitioners detect and reduce order-manipulation risk.

References

- Qingyao Ai, Keping Bi, Cheng Luo, Jiafeng Guo, and W. Bruce Croft. 2018. Unbiased learning to rank with unbiased propensity estimation. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*.
- Ethan Bito, Yongli Ren, and Estrid He. 2025. Evaluating position bias in large language model recommendations. *arXiv preprint arXiv:2508.02020*.
- Ethan Bito, Yongli Ren, and Estrid He. 2026. One pass, any order: Position-invariant listwise reranking for llm-based recommendation. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM.
- Christopher J.C. Burges. 2010. From RankNet to LambdaRank to LambdaMART: An overview. Technical Report MSR-TR-2010-82, Microsoft Research.
- Wen-Shuo Chao, Zhi Zheng, Hengshu Zhu, and Hao Liu. 2024. Make large language model a better ranker. In *Findings of the Association for Computational Linguistics: EMNLP 2024*.
- Huiyuan Chen, Yusan Lin, Menghai Pan, Lan Wang, Chin-Chia Michael Yeh, Xiaoting Li, Yan Zheng, Fei Wang, and Hao Yang. 2022. Denoising self-attentive sequential recommendation. In *Proceedings of the 16th ACM conference on recommender systems*.
- Shijie Chen, Bernal Jiménez Gutiérrez, and Yu Su. 2025. Attention in large language models yields efficient zero-shot re-rankers. In *Proceedings of the International Conference on Learning Representations (ICLR 2025)*.
- Xinyun Chen, Ryan A Chi, Xuezhi Wang, and Denny Zhou. 2024. Premise order matters in reasoning with large language models. *arXiv preprint arXiv:2402.08939*.
- Beni Egressy and Jan Stühmer. 2026. Set-llm: A permutation-invariant llm. *Advances in Neural Information Processing Systems*, 38:62798–62834.
- Philipp Hager, Romain Deffayet, Jean-Michel Renders, Onno Zoeter, and Maarten de Rijke. 2024. Unbiased learning to rank meets reality: Lessons from baidu’s large-scale search dataset. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1546–1556.
- F. Maxwell Harper and Joseph A. Konstan. 2015. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19.
- Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yongdong Zhang, and Meng Wang. 2020. Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 639–648.
- Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiusi Chen, and Julian McAuley. 2024a. Bridging language and items for retrieval and recommendation. In *arXiv preprint arXiv:2403.03952*.
- Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024b. Large language models are zero-shot rankers for recommender systems. In *European Conference on Information Retrieval*, pages 364–381.
- Hongye Jin, Xiaotian Han, Jingfeng Yang, Zhimeng Jiang, Zirui Liu, Chia-Yuan Chang, Huiyuan Chen, and Xia Hu. 2024. Llm maybe longlm: Selfextend llm context window without tuning. In *Forty-first International Conference on Machine Learning*.
- Thorsten Joachims, Adith Swaminathan, and Tobias Schnabel. 2017. Unbiased learning-to-rank with biased feedback. In *Proceedings of the Tenth ACM*

- International Conference on Web Search and Data Mining.*
- Sachin Kumar and Himabindu Lakkaraju. 2024. Manipulating large language models to increase product visibility. *arXiv preprint arXiv:2404.07981*.
- Zongjie Li, Chaozheng Wang, Pingchuan Ma, Daoyuan Wu, Shuai Wang, Cuiyun Gao, and Yue Liu. 2024. Split and merge: Aligning position biases in llm-based evaluators. *arXiv preprint arXiv:2310.01432*.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024a. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Yu-An Liu et al. 2024b. Attack-in-the-chain: Bootstrapping large language models for attacks against black-box neural ranking models. In *Proceedings of the AAAI Conference on Artificial Intelligence*.
- Liang-bo Ning et al. 2024. Cheatagent: Attacking recommender systems via llm agent. *arXiv preprint arXiv:2409.10002*.
- Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. 2023a. Rankvicuna: Zero-shot listwise document reranking with open-source large language models. *arXiv preprint arXiv:2309.15088*.
- Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. 2023b. Rankzephyr: Effective and robust zero-shot listwise reranking is a breeze! *arXiv preprint arXiv:2312.02724*.
- Hongjin Qian et al. 2025. Jailbreaking llm-based rankers via task substitution and ranking criterion manipulation. *arXiv preprint arXiv:2510.10003*.
- Jingfen Qiao, Jin Huang, Xinyu Ma, Shuaiqiang Wang, Dawei Yin, Evangelos Kanoulas, and Andrew Yates. 2026. Llm-based listwise reranking under the effect of positional bias. In *Advances in Information Retrieval, Lecture Notes in Computer Science*. Springer.
- Revanth Gangi Reddy, JaeHyeok Doo, Yifei Xu, Md Arafat Sultan, Deevya Swain, Avirup Sil, and Heng Ji. 2024. First: Faster improved listwise reranking with single token decoding. *arXiv preprint arXiv:2406.15657*.
- Lin Shi, Chiyu Ma, Weicheng Liang, Xingjian Diao, Weining Ma, and Sadeh Vosoughi. 2025. Judging the judges: A systematic study of position bias in llm-as-a-judge. *arXiv preprint arXiv:2406.07791*.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063.
- Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is chatgpt good at search? investigating large language models as re-ranking agents. *arXiv preprint arXiv:2304.09542*.
- Raphael Tang, Crystina Zhang, Xueguang Ma, Jimmy Lin, and Ferhan Ture. 2024. Found in the middle: Permutation self-consistency improves listwise ranking in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2024)*.
- Yiming Tang et al. 2025. Stealthrank: Llm ranking manipulation via stealthy prompt optimization. *arXiv preprint arXiv:2504.05804*.
- Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhifang Sui. 2023. Large language models are not fair evaluators. *arXiv preprint arXiv:2305.17926*.
- Xuanhui Wang, Cheng Li, Nadav Golbandi, Michael Bendersky, and Marc Najork. 2018. The LambdaLoss framework for ranking metric optimization. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 1313–1322.
- Zongwei Wang et al. 2024. Textsimu: Llm-driven n-persona simulation attack for recommendation exposure. *arXiv preprint arXiv:2409.10001*.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2023. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*.
- Tiancheng Xing, Jerry Li, Yixuan Du, and Xiyang Hu. 2026. Are llms reliable rankers? rank manipulation via two-stage token optimization. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9120–9132.
- Soyoung Yoon, Eunbi Choi, Jiyeon Kim, Hyeon-gu Yun, Yireun Kim, and Seung-won Hwang. 2024. Listt5: Listwise reranking with fusion-in-decoder improves zero-shot retrieval. *arXiv preprint arXiv:2402.15838*.
- Yifan Zeng, Ojas Tendolkar, Raymond Baartmans, Qingyun Wu, Lizhong Chen, and Huazheng Wang. 2024. Llm-rankfusion: Mitigating intrinsic inconsistency in llm-based ranking. *arXiv preprint arXiv:2406.00231*.
- Zheng Zhang, Fan Yang, Ziyang Jiang, Zheng Chen, Zhengyang Zhao, Chengyuan Ma, Liang Zhao, and Yang Liu. 2024. Position-aware parameter-efficient fine-tuning approach for reducing positional bias in llms. *arXiv preprint arXiv:2404.01430*.

Yuying Zhao, Xiaodong Yang, Huiyuan Chen, Xiran Fan, Yu Wang, Yiwei Cai, and Tyler Derr. 2025. Simaug: Enhancing recommendation with pre-trained language models for dense and balanced data augmentation. *arXiv preprint arXiv:2505.01695*.

Shengyao Zhuang, Honglei Zhuang, Bevan Koopman, and Guido Zuccon. 2024. A setwise approach for effective and highly efficient zero-shot ranking with large language models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*.

A Additional Results and Diagnostics

A.1 Dataset Construction Details

Label construction. Candidate labels come from temporally held-out future interactions rather than manual ranking annotations. Observed ratings map to graded relevance labels under a shared rule: ratings 1, 2, 3, and 4/5 become labels 1, 2, 3, and 4 respectively, so the top two rating levels are collapsed into a single highest-relevance label. Retrieved negatives and catalog filler items which sampled from the catalog to keep the candidate-set size fixed, receive label 0. Label 0 therefore denotes a non-positive candidate under the evaluation split, while labels above 0 denote observed future interactions with graded relevance.

Inserting the held-out positives directly is necessary in the Amazon domains rather than a convenience. Requiring LightGCN to retrieve the held-out future item—the `REQPOS=true` condition—would discard most otherwise usable evaluation instances, because first-stage `Recall@100` is only about 1.3% on both Amazon domains (Table 3). We therefore use `REQPOS=false` throughout. Because the positive is inserted at a random position rather than at a retrieval-determined one, its input position carries no information about relevance, which is a precondition for attributing rank changes to presentation order alone.

Ground-truth ranking and ties. The ground-truth ranking is induced by decreasing relevance label. Candidates with equal labels are treated as ties, and the LambdaRank objective (Borges, 2010; Wang et al., 2018) is applied only to item pairs with unequal labels. Table 3 reports raw dataset sizes, filtering thresholds, split sizes, history windows, and first-stage retrieval recall for all three domains.

A.2 Prompt-Level Mitigation Does Not Close the Attack Surface

A natural first reaction to the position-bias attack is to simply *instruct* the reranker to ignore candidate order. We test whether this works. Holding the *same* trained causal reranker fixed (same weights, causal attention, standard RoPE), we change *only* the eval-time instruction and re-run both probes—the position scan (`CURVE_RANGE`) and the budget- R permutation attacker (`promo@5`). If a prompt genuinely removes the bias, its metrics should collapse toward 0, matching the architectural / consistency-trained defenses; if they stay at the baseline, the bias is *mechanistic* rather than *instructional*.

All prompts share the listwise template below; only the `INSTRUCTION` slot varies as shown in Table 4. Candidate and history formatting are byte-identical across variants, so wording is the sole moving part.

Shared listwise template

```
[SPAN]
<Instruction>
User history:
title: ...
[/SPAN]
[ITEM] <candidate 1> [/ITEM]
[ITEM] <candidate 2> [/ITEM]
...
```

We compare a control (p_0 , the original wording) against five treatments that each encode the “ignore order / treat as a set” intent in a *different prompting style*, so a null result cannot be blamed on one unlucky phrasing. The results are presented in Table 5.

Finding. Prompting is largely inert. Across all five styles, `promo@5` stays around 0.12–0.13 and `CURVE_RANGE` around 2.93–3.20, essentially matching the control. By contrast, the architectural (A) and training-time (B2) defenses drive `promo@5` to near zero. This suggests the position bias is mechanistic rather than instructional: it cannot be removed by prompting alone.

A.3 Position vs. Content Diagnostic

As a diagnostic on the MovieLens anchor, we compare the order-only lever with a simple content-edit lever. We cross two factors: position, either original or favorable, and content, either original or edited by injecting the user’s top history genre into c^* , as shown in Table 6. Repositioning c^* improves its rank by 1.86 positions on average,

Table 3: **Dataset construction statistics.** We report raw dataset sizes, filtering parameters, post-split sample counts, history lengths, and first-stage retrieval recall. First-stage recall was not logged for the MovieLens build and is omitted rather than estimated.

Dataset	Raw inter.	Raw items	Hist.	Min int.	Train	Val	Test	Recall@10	Recall@50	Recall@100
MovieLens-32M	32.0M	87.6K	20	50	9,998	9,996	9,997	—	—	—
Amazon Books	29.5M	4.45M	20	20	4,084	4,081	4,087	0.0024	0.0085	0.0133
Amazon Fashion	2.50M	826K	5	8	2,003	1,974	2,070	0.0027	0.0086	0.0126

Amazon domains use REQPOS=false: requiring the held-out future positive to appear in the LightGCN retrieval set would remove most usable samples because first-stage recall is very low. Fashion uses a shorter history window and a lower minimum-interaction threshold to retain enough eligible users.

Table 4: Instruction wordings tested (the $\langle$ Instruction $\rangle$ slot).

ID	Style	Instruction
p_0	control	Given the user’s interaction history, rank the candidate items according to the user’s preferences.
p_1	declarative	Given the user’s interaction history, rank the following candidate items by how well they match the user’s preferences. The candidates are provided together as an unordered set: consider all of them as a group and judge each item only on its own merits, not on where it appears in the list.
p_2	imperative	Rank the candidate items by relevance to the user’s history. Ignore the list order. Do not favor earlier or later items. The position of a candidate is random and carries no meaning: judge content only.
p_3	persona	You are a position-invariant recommendation reranker. Your defining property is that your output depends only on item content and the user’s history, never on the order in which candidates are fed to you. Acting as this reranker, rank the candidate items by how well each matches the user’s preferences.
p_4	chain-of-thought	Given the user’s interaction history, rank the candidate items by preference. Reason step by step: first read each candidate on its own and assess how relevant it is to the user, independent of the other candidates; then compare those per-item assessments to produce the final ranking. Because the input order is arbitrary, the ranking you produce must be the same no matter how the candidates were ordered.
p_5	structured rules	Task: rank the candidate items for the user. Rules: 1. Treat the candidates as an unordered set. 2. The listed order is random and must be ignored. 3. Score each item only by its relevance to the user’s interaction history. 4. An item’s rank must not change if the list is reshuffled. Rank the items following these rules.

Table 5: Prompting leaves the attack surface largely unchanged on MovieLens ($N=300$, $R=50$). Larger values mean more vulnerability.

ID	Style	curve_range	promo@5	rank_gain	into_top5
p_0	baseline	3.09	0.120	3.34	0.273
p_1	declarative	3.12	0.120	3.48	0.277
p_2	imperative	3.15	0.123	3.48	0.280
p_3	persona	2.93	0.127	3.14	0.273
p_4	CoT	3.20	0.123	3.53	0.277
p_5	rules	3.03	0.130	3.40	0.280
architectural invariance (A)		0.06	0.000	—	—
consistency-KL (B2)		—	0.007	—	—

whereas this content edit improves it by 0.33 positions, with negligible interaction between the two effects. This suggests that candidate position can act as a strong content-free lever in this setting. We report this diagnostic to separate positional effects from content effects, not as a comprehensive comparison against optimized content attacks.

A.4 Additional Defense Diagnostics

Table 7 reports the full anchor defense matrix on MovieLens, including mask-only invariance,

Table 6: **Position-vs.-content diagnostic on the MovieLens anchor.** Rank improvement from repositioning a label-0 target versus applying a simple relevance-signal content edit. Higher gain means stronger promotion.

Intervention	Mean rank gain
Favorable position only	1.86
Content edit only	0.33
Interaction	negligible

position-only invariance, order-augmented training, permutation-consistency regularization, test-time averaging, and pointwise scoring. Table 8 reports the corresponding cross-domain stability diagnostics, including Kendall’s τ and label-0 position-curve range.

A.5 Full Cross-Model Sweep

Table 9 reports the complete cross-model and cross-domain sweep corresponding to the scale and domain analysis in Section 7. The main text

Table 7: **Full anchor defense matrix on MovieLens.** This table reports all defense variants evaluated on the Llama-3.2-3B anchor. Lower promo@5 is better. Exact indicates whether the method is invariant by construction.

Defense / model	nDCG@10	promo@5	Cost	Exact
Baseline causal	0.829	0.120	1×	No
A: Architectural invariance	0.784	0.000	1×	Yes
A1: Mask-only invariance	0.783	0.070	1×	No
A2: Position-only invariance	0.769	0.090	1×	No
B1: Order-augmented training	0.849	0.117	1×	No
B2: Permutation-consistency KL	0.777	0.007	1×	Approx.
C: Test-time averaging ($P = 20$)	0.848	≈ 0	20×	No
Pointwise reference	0.763	0.000	$K \times$	Yes

For test-time averaging, promo@5 ≈ 0 is inferred from averaged-score behavior rather than re-running the full adversarial search against the averaged scorer.

Table 8: **Full cross-domain defense diagnostics.** In addition to ranking quality and manufactured exposure, we report permutation stability τ and label-0 position-curve range. Higher τ and lower range indicate greater order stability.

Domain	Defense / model	nDCG@10	promo@5	τ	Range	Exact
MovieLens	Baseline causal	0.829	0.120	0.845	3.09	No
MovieLens	Architectural invariance	0.784	0.000	0.992	0.06	Yes
MovieLens	Permutation-consistency KL	0.777	0.007	0.983	0.21	Approx.
MovieLens	Pointwise reference	0.763	0.000	1.000	0.00	Yes
Amazon Books	Baseline causal	0.618	0.480	0.563	6.62	No
Amazon Books	Architectural invariance	0.510	0.000	0.990	0.06	Yes
Amazon Books	Permutation-consistency KL	0.552	0.067	0.912	1.16	Approx.
Amazon Books	Pointwise reference	–	0.000	1.000	0.00	Yes
Amazon Fashion	Baseline causal	0.449	0.297	0.661	4.12	No
Amazon Fashion	Architectural invariance	0.424	0.003	0.984	0.06	Yes
Amazon Fashion	Permutation-consistency KL	0.424	0.047	0.934	0.65	Approx.
Amazon Fashion	Pointwise reference	–	0.000	1.000	0.00	Yes

summarizes these results by domain-level ranges; here we provide the full per-model values.

A.6 Audit-Correlation Robustness

Table 10 reports robustness checks for the relationship between ordinary permutation stability and manufactured exposure. The negative relationship between Kendall’s τ and promo@5 remains strong under leave-one-domain-out checks and after controlling for domain identity.

A.7 Attack-Budget Values

Table 11 provides the numerical values behind the attack-budget curve in Figure 6. The causal reranker becomes more exploitable as the attacker tries more candidate orderings, while construction-based invariant references remain at zero.

Table 9: **Full cross-model and cross-domain causal sweep.** For each causal reranker, we report standard ranking quality using nDCG@10 and manufactured exposure using promo@5 at attack budget $R = 50$. Across the evaluated models, promo@5 does not monotonically decrease with parameter count; domain identity shifts the attack surface more strongly than scale. The final row reports the position-invariant reference on the anchor model.

Model	MovieLens		Amazon Books		Amazon Fashion	
	nDCG@10	promo@5	nDCG@10	promo@5	nDCG@10	promo@5
Qwen3-0.6B	0.791	0.130	0.534	0.497	0.398	0.353
Qwen3-1.7B	0.751	0.100	0.565	0.427	0.425	0.227
Qwen3-4B	0.798	0.100	0.597	0.497	0.432	0.387
Qwen3-8B	0.817	0.097	0.616	0.570	0.435	0.317
Qwen3-14B	0.829	0.140	0.633	0.477	0.433	0.273
Llama-3.1-8B	0.818	0.113	0.610	0.497	0.418	0.270
Mistral-7B	0.865	0.147	0.646	0.540	0.448	0.267
Llama-3.2-3B	0.829	0.120	0.618	0.480	0.449	0.297
Position-invariant reference	0.784	0.000	0.510	0.000	0.424	0.003

Table 10: **Audit-correlation robustness checks.** We correlate causal-reranker permutation stability τ with manufactured exposure promo@5. The relationship remains strong after controlling for domain identity and under leave-one-domain-out checks.

Analysis	n	Pearson r	Pearson p	Spearman ρ	Spearman p
All 24 cells	24	-0.974	$< 10^{-4}$	-0.969	$< 10^{-4}$
Amazon Books only	8	-0.857	0.0066	-0.683	0.0618
Amazon Fashion only	8	-0.812	0.0143	-0.786	0.0208
MovieLens only	8	-0.541	0.1665	-0.683	0.0621
Leave out Amazon Books	16	-0.964	$< 10^{-4}$	-0.935	$< 10^{-4}$
Leave out Amazon Fashion	16	-0.993	$< 10^{-4}$	-0.923	$< 10^{-4}$
Leave out MovieLens	16	-0.971	$< 10^{-4}$	-0.935	$< 10^{-4}$
Partial corr., control domain	24	-0.776	8×10^{-6}	—	—

Table 11: **Attack-budget sensitivity on the MovieLens anchor.** promo@5 is reported for label-0 targets, from the same $R_{\max} = 50$ reference run as Table 1.

Model	$R = 1$	$R = 5$	$R = 10$	$R = 20$	$R = 50$
Causal Llama-3B	0.023	0.067	0.090	0.103	0.120
T5 encoder scorer	0.007	0.030	0.040	0.047	0.057
Architectural invariance	0.000	0.000	0.000	0.000	0.000
Pointwise reference	0.000	0.000	0.000	0.000	0.000